

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

PRÁCTICA I — SORTING LARGE DATASET

ST0245 – SI001 · Estructuras de Datos y Algoritmos

Docente: Carlos Alberto Alvarez Henao
Estudiante: Daniela Giraldo Salas
Medellín, Colombia · marzo 2026

1. Introducción

El presente informe documenta el desarrollo e implementación de la **Práctica I** del curso ST0245 – Estructuras de Datos y Algoritmos de la Universidad EAFIT. El objetivo principal es analizar el comportamiento de tres estrategias de ordenamiento distintas aplicadas sobre un dataset de **100,000 palabras** en inglés.

Las tres estrategias implementadas son:

- QuickSort sobre un arreglo dinámico (`std::vector<string>`)
- HeapSort utilizando un Binary Max-Heap
- Árbol AVL con inserción y recorrido inorden

Cada implementación fue desarrollada desde cero en C++17, sin el uso de funciones de ordenamiento predefinidas de la librería estándar. El análisis contempla tiempo de ejecución real, uso estimado de memoria y complejidad algorítmica teórica.

2. Dataset

El dataset fue proporcionado por el docente y consiste en 100,000 palabras en inglés en orden aleatorio, almacenadas en el archivo `dataset.txt`. El cual fue recibido en orden aleatorio, garantizando condiciones de entrada equitativas para todos los algoritmos."

2.1 Características del Dataset

Característica	Detalle
Fuente	Proporcionado por el docente
Total de palabras	100,000
Tipo de dato	<code>std::string</code> (palabras en minúscula)
Estado inicial	Recibido en orden aleatorio
Archivo de entrada	<code>dataset.txt</code>

3. Enfoque de Implementación

3.1 QuickSort

QuickSort fue implementado de forma iterativa utilizando una pila explícita (`std::vector<pair<int,int>>`) en lugar de recursión. Esta decisión evita el desbordamiento de pila (stack overflow) que puede ocurrir con 100,000 elementos en la versión recursiva clásica.

Adicionalmente, se empleó la técnica de mediana de tres para la selección del pivote: se compara el elemento inicial, el del medio y el final, y se escoge el valor mediano. Esto reduce significativamente el riesgo del peor caso $O(n^2)$ que ocurre cuando el dataset está completamente ordenado.

Estructura de datos: `std::vector<string>` — arreglo dinámico contiguo en memoria.

Ventaja principal: excelente localidad de caché por acceso secuencial a memoria.

3.2 HeapSort

HeapSort fue implementado construyendo un Binary Max-Heap in-place sobre el mismo vector. El proceso consiste en dos fases: primero se convierte el arreglo en un heap válido mediante llamadas a heapify desde el último nodo padre hasta la raíz, y luego se extraen los elementos intercambiando la raíz con el último elemento y reduciendo el tamaño del heap en cada paso.

Estructura de datos: `std::vector<string>` usado como representación de árbol binario (índice $i \rightarrow$ hijos $2i+1$ y $2i+2$).

Ventaja principal: garantía de $O(n \log n)$ en todos los casos, sin necesidad de estructuras auxiliares.

3.3 Árbol AVL

El Árbol AVL fue implementado con inserción balanceada que verifica el factor de balance en cada nodo tras cada inserción y aplica las rotaciones necesarias (LL, RR, LR, RL) para mantener el árbol balanceado. El recorrido inorden fue implementado de forma iterativa usando una pila explícita para evitar recursión profunda con 100,000 nodos.

Estructura de datos: árbol binario de búsqueda balanceado, cada nodo contiene: clave (string), punteros left/right y altura.

Ventaja principal: permite búsqueda e inserción en $O(\log n)$ garantizado, útil si se necesitan consultas posteriores al ordenamiento."

4. Mediciones de Rendimiento

Resultados obtenidos al ejecutar los tres algoritmos sobre el dataset de 100,000 palabras en la misma ejecución del programa

Algoritmo	Tiempo (ms)	Memoria (KB)	¿Ordenado?
QuickSort	32.33	4,595	SÍ
HeapSort	63.69	4,595	SÍ
Árbol AVL	68.94	6,488	SÍ

4.1 Estimación de Memoria

La estimación de memoria se realizó usando `sizeof()` sobre las estructuras utilizadas y sumando el espacio ocupado por cada elemento almacenado:

- QuickSort y HeapSort: se calculó el tamaño del `std::vector` más la capacidad de cada `std::string` almacenada (~4,595 KB).
- Árbol AVL: se estimó el tamaño de cada `AVLNode` (punteros `left/right`, campo `height`, objeto `string`) multiplicado por el número de nodos únicos (~6,488 KB).

El mayor consumo del Árbol AVL (~1,893 KB adicionales frente a los métodos basados en vector) se explica por el overhead de cada asignación dinámica individual en el heap del sistema operativo. más los tres campos extra de control por nodo (dos punteros y la altura).

5. Comparación entre Algoritmos

5.1 Complejidad Algorítmica (Big O)

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Espacio
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Árbol AVL	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

5.2 ¿Cuál algoritmo tuvo mejor desempeño?

QuickSort fue el más rápido con 32.33 ms, aproximadamente el doble de velocidad que los otros dos algoritmos. Aunque su peor caso teórico es $O(n^2)$, en la práctica supera a los demás gracias a su excelente **localidad de caché**: su patrón de acceso a memoria es predominantemente secuencial durante la partición, lo que minimiza los fallos de caché.

5.3 ¿Por qué la complejidad teórica difiere de los resultados prácticos?

QuickSort y HeapSort tienen la misma complejidad promedio $O(n \log n)$, pero HeapSort tardó casi el doble. Esto ilustra que la complejidad Big O no captura los factores de hardware modernos:

- Localidad de caché: QuickSort accede a memoria de forma secuencial. HeapSort salta entre posiciones distantes (índice $i \rightarrow 2i+1, 2i+2$), generando más fallos de caché.
- Overhead de asignación: el Árbol AVL realiza una llamada a `new` por cada uno de los 100,000 nodos. Estas asignaciones dinámicas son costosas en tiempo real aunque no aparecen en el análisis Big O.
- Constante oculta: dos algoritmos con la misma O pueden diferir por 2x o 3x en práctica según implementación, tamaño de datos y arquitectura del procesador.

5.4 Ventajas y desventajas de cada estructura

`std::vector` (QuickSort / HeapSort):

- **Ventaja:** almacenamiento contiguo, acceso $O(1)$ por índice, óptimo para caché.
- **Desventaja:** no permite búsquedas eficientes una vez ordenado sin índice adicional.

Árbol AVL:

- **Ventaja:** búsqueda e inserción en $O(\log n)$ garantizado, útil para consultas y actualizaciones frecuentes tras el ordenamiento inicial.
- **Desventaja:** mayor uso de memoria (punteros + altura por nodo), mayor tiempo de construcción por las rotaciones de balanceo.

6. Conclusiones

Para el problema de ordenamiento puntual de un dataset estático de 100,000 palabras, **QuickSort es la estrategia más apropiada**: es el más rápido, opera in-place (menor uso de memoria) y su implementación iterativa con mediana de tres lo hace robusto en la práctica.

HeapSort es la mejor alternativa cuando se necesitan garantías estrictas de tiempo de ejecución: su $O(n \log n)$ en el peor caso es una ventaja real en sistemas críticos donde la variabilidad de QuickSort no es aceptable.

El Árbol AVL, aunque es la opción más lenta y que más memoria consume para un ordenamiento puntual, se vuelve la estructura más poderosa si el problema incluye consultas frecuentes después del ordenamiento, ya que permite búsquedas en $O(\log n)$ sin necesidad de estructuras adicionales.

Este ejercicio demuestra que la complejidad algorítmica teórica es una guía necesaria pero insuficiente: el comportamiento real depende también de la arquitectura del hardware, los patrones de acceso a memoria y los costos de asignación dinámica.